University
Mohammed VI
Polytechnic

College of
Computing

Deliverable #:Lab 4

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2026

Team Information

Members Names	Roles
TAHA YASSINE	Part1: People & Laboratories
SAAD SABER	Part2: Research Projects & Reservations
HAFSA SAHNOUNE	Part3: Equipment & Maintenance
ACHRAF RHANNAM	Part4: Consumables & Stock
Repository Link	`https://github.com/...`

---

1 List of FDs

1.1 Part1:

1. Person:
   $pid \rightarrow firstName, lastName, email, affiliation, startDate, endDate$
   Justification: pid is the primary key of Person and determines all its non-key attributes

2. Student:
   $pid \rightarrow program$
   justification: pid is the primary key of Student and determines its non-key attribute

3. Employee:
   $pid \rightarrow phone, office, supervisorPid$
   justification: pid is the primary key of Employee and determines all its non-key attributes.

4. Academic:
   No non-trivial FDs exist because the relation contains only its primary key

5. NonAcademic:
   No non-trivial FDs exist because the relation contains only its primary key

6. Faculty:
   $pid \rightarrow position$
   justification: pid is the primary key of Faculty and determines position

7. Administrative:
   $pid \rightarrow position$
   Justification: pid is the primary key of Administrative and determines position

8. Technical:
   pid $\rightarrow position$
   Justification: pid is the primary key of Technical and determines position.

9. Advises:
   No non-trivial FDs exist because the relation contains only its composite primary key.

10. Laboratory:
    $labId \rightarrow name, building, roomNumber, discipline, supervisorPid$
    Justification: labId is the primary key of Laboratory and determines all its non-key attributes.

11. Attached:
    No non-trivial FDs exist because the relation contains only its composite primary key.

1.2 Part2:

12. **Table** `ResearchProject`**:**
    projCode $\rightarrow$ title, startDate, endDate, status
    Justification (PR1): `ResearchProject` is a strong entity. Its primary key (`projCode`) uniquely determines all of its scalar attributes.

13. **Table** `ParticipatesIn`**:**
projCode, pid → role
Justification (PR2–PR4): Mapped from the M:N relationship between `Person` and `ResearchProject`. Since a person plays exactly one role in each project (PR4), the composite key determines `role`. The converse projCode, role → pid does not hold, since a project may have many collaborators. The "exactly one PI" rule (PR3) is a cardinality constraint, not an FD.

14. **Table** `Budget`**:**
budgetLine → amountGranted, amountDisbursed, startDate, endDate, managerPid
Justification (BD1, BD2): The budgetary line identifies a budget and determines its amounts and dates. The `Manages` relationship is absorbed, since each budget is managed by exactly one academic employee, so `budgetLine` determines `managerPid`.

15. **Table** `FundsProject`**:**
Only trivial FDs (all attributes form the key: budgetLine, projCode).
Justification (BD3): M:N relationship with no descriptive attributes, so there is no non-trivial FD. "At least one budget per project" is a participation constraint.

16. **Table** `FundsLab`**:**
Only trivial FDs (all attributes form the key: budgetLine, labId).
Justification (BD3): Same reasoning as `FundsProject`.

17. **Table** `Reservation`**:**
resId → submissionTs, plannedStart, plannedEnd, purpose, status, makerPid, projCode, approverP
Justification (RE1–RE3): `resId` is the primary key (RE2). Each reservation is made by exactly one person for one project (RE1), so `resId` determines `makerPid` and `projCode`. A reservation has at most one approver (RE3), so `resId` determines `approverPid`, which may be `NULL`.

18. **Table** `Reserves`**:**
Only trivial FDs (all attributes form the key: resId, serialNo).
Justification (RE1, RE4): M:N relationship between `Reservation` and `EquipmentUnit` with no descriptive attributes.

1.3 Part3:

19. *EquipmentUnit* :
serialNo → acquisitionDate, purchaseCost, status, portable, modelId, labId
Justification: the serial number identifies a unit, which has one value for each attribute (EQ3, EQ6). It belongs to exactly one model (EQ5) and only its current lab is stored (EQ4).

20. *EquipmentModel* :
modelId → commercialName, manufacturer, category,
requiredEnvironment, trainingMandatory
Justification: the model identifier determines all the properties stored for a model (EQ2).

21. *Certification* :
code → title, issuingAuthority, validityPeriod, safetyLevel
Justification: the code identifies a certification, which has a single title, authority, validity period and safety level (CE2).

22. *Requires* :
(modelId, certCode) →
Justification: a model can require several certifications and a certification can cover several models (CE6), so neither attribute determines the other.

23. *Holds* :
(pid, certCode) → issueDate, expirationDate, grade
Justification: only the latest award of a certification is kept (CE5), so each (person, certification) pair has one issue date, one expiration date and at most one grade (CE4).

24. *Maintenance* :
(serialNo, startTS) → endTS, type, description, cost, outcome, technicalPid
Justification: a unit can be maintained many times (MA1), so an operation is identified by its unit and start time (MA2), and it is carried out by exactly one technician (MA3).

25. *CalibrationRecord* :
(serialNo, calibDate) → calibrationType, result, nextDueDate, remarks
Justification: a record belongs to one unit (CA3) and is identified within it by its calibration date (CA4).

1.4 Part4 :

26. **Table `Consumable`:**
consId → name, unitOfMeasure, hazardLevel, reorderThreshold
Justification: `Consumable` is a strong entity. Its primary key (`consId`) functionally determines all of its scalar attributes.

27. **Table `Supplier`:**
suppId → name, contactEmail, phone
Justification: `Supplier` is a strong entity. Its primary key (`suppId`) uniquely determines all associated contact details.

28. **Table `Supplies`:**
suppId, consId, labId → unitPrice
Justification: Mapped from a ternary relationship in Lab 1. The composite key determines `unitPrice` since prices vary depending on the specific supplier, consumable, and laboratory combination.

29. **Table `Stocks`:**
labId, consId → quantityOnHand, lastRestockDate, storageCondition, technicalPid, monitoringSin
Justification: Mapped from an M:N relationship with a composite key. Since Lab 1 dictates each stock occurrence is monitored by exactly one technician, this key also determines `technicalPid`.

30. **Table `Consumes`:**
resId, labId, consId → quantityUsed
Justification: Connects a `Reservation` to a specific `Stocks` occurrence. The composite key strictly determines the `quantityUsed` for that exact transaction.

2 FDs based on assumptions and challenges :

2.1 Part1:

Challenges: The main challenge was identifying FDs separately for each relation, especially those involved in ISA hierarchies. We distinguished functional dependencies from foreign-key constraints and handled relations with composite primary keys but no non-key attributes.
Assumptions: Primary keys uniquely identify tuples, and non-key attributes are determined by their relation's primary key. ISA relationships are represented through foreign keys, which are documented separately from the FD list.

2.2 Part2:

1. **Absorbed relationships:** `Manages`, `Makes`, `ForProject` and `Approves` are many-to-one, so they are merged into `Budget` and `Reservation`.

2. **Not expressible as FDs:** participation constraints (PR2, BD3, RE1), role eligibility rules, and the CE7 certification check.

2.3 Part3:

3. *EquipmentModel* :
(commercialName, manufacturer) → *modelId*
Assumption: a manufacturer never uses the same commercial name for two different models, so this pair is a second candidate key.

4. *Maintenance* :
(serialNo, endTS) → *startTS*
Assumption: operations on the same unit never overlap in time, so two of them cannot end at the same moment.

2.4 Part4:

5. **Absorbing `Monitors` (`Stocks`):** Translated the ER aggregation by placing technicalPid functionally dependent on labId, consId, reflecting the exactly-one mapping without needing a separate relation.

6. **Ternary Dependency (`Supplies`):** Maintained unitPrice dependence on the full triplet suppId, consId, labId to accurately map the Lab 1 ternary relationship.